

O vocabulário formal e o trabalho empírico

Nos *Sprints* 1 a 5 vocês produziram seis artefatos dentro do diretório `internet_banking_tests` sem necessariamente saber, no momento da produção, qual nome formal cada um deles teria no vocabulário do BSTQB CTFL v4.0 (BSTQB, 2023). Isso não é um problema: é a sequência natural de aprendizagem por imersão — primeiro a prática crua, depois o nome técnico. Mas avançar do *Sprint* 5 para o *Sprint* 7 sem fechar o nome é deixar o trabalho órfão de léxico, e o léxico é justamente o que permite conversar com banca, com auditoria e com colegas que vieram de outras trajetórias acadêmicas.

Por que dar nomes agora

Três razões objetivas:

- i) a prova teórica da disciplina exige reconhecimento dos termos do BSTQB para os níveis e técnicas de teste;
- ii) a culminância da Hackathon em 26-06-2026 vai exigir que sua equipe descreva, na linguagem da banca, o que cada arquivo de teste do repositório faz e a qual estratégia ele responde; e
- iii) o plano de ensino prevê a Unidade 2 — Níveis, Tipos e Técnicas de teste — como bloco formal a ser fechado. Re-rotular agora encerra essa unidade sem reensino e libera as três semanas finais do semestre para as Unidades 3 (especificação estruturada de casos de teste) e 7 (introdução à maturidade).

Conceito essencial: a virada deste sprint

Nos *Sprints* 1 a 3 a pergunta foi: o sistema responde corretamente?

No *Sprint* 4: minha suíte detectaria se ele parasse de responder?

No *Sprint* 5: meus mutantes sobreviventes são equivalentes ou matáveis?

Neste *Sprint* 6 a pergunta muda mais uma vez, e o salto agora é de ordem epistemológica e profissional: que nome formal cada artefato meu tem na taxonomia do BSTQB, e o que muda na minha capacidade de comunicar engenharia quando eu sei o nome?

Não há código novo neste sprint. Há leitura crítica e rotulação técnica do que já existe na sua pasta de trabalho.

Teste de Software

Roteiro Sprint 6 — re-rotulação semântica dos artefatos dos Sprints 1 a 5

Como ler este roteiro

Este roteiro tem uma lógica de progressão. Cada bloco prepara o seguinte. Você vai encontrar blocos de cinco naturezas, e precisa tratá-los de formas diferentes:

Blocos azuis trazem conceito ou continuidade. Leia com atenção; não há resposta a registrar.

Blocos rosa trazem investigação técnica do aluno. Você precisa registrar sua resposta no arquivo Word de entrega antes de avançar. A resposta é avaliada por consistência com os autores da disciplina (BSTQB, 2023; COUTINHO; NASCIMENTO, 2025; JIA; HARMAN, 2011; MACIVER et al., 2019; PAPADAKIS et al., 2019; RAHMAN et al., 2024), não por concordância com nenhum gabarito.

Blocos amarelos trazem atenção técnica ou troubleshooting.

Blocos roxos trazem comandos ou prompts para copiar. NUNCA cole o comando antes de entender o que ele faz.

Blocos verdes trazem resultado esperado ou reflexão final. Eles, são especialmente importantes — é como você sabe que está no caminho certo sem o professor ao lado.

Atenção: respostas sem ancoragem nos autores da disciplina, sem ancoragem nos artefatos efetivamente produzidos nos *Sprints 1 a 5* ou construídas apenas com definições genéricas não serão consideradas tecnicamente válidas.

Objetivo deste sprint

Você não vai escrever código novo neste sprint. Vai abrir os arquivos que já produziu — `test_ia_gerado.py` (Sprint 1), `confest.py` (Sprint 2), `test_hypothesis.py` (Sprint 3), o `app.py` refatorado a partir de `app2.py` com `config.py` externo e o `test_flask_client.py` (Sprint 4), e `features/transferencia.feature` com `features/steps/transferencia_steps.py` (Sprint 5) — e atribuir a cada um o seu nome formal segundo o vocabulário do BSTQB CTFL v4.0 (BSTQB, 2023, Cap. 2 e Cap. 4). O ganho não é técnico no sentido de novas linhas de código; é epistêmico e profissional: você sai do *Sprint 6* capaz de descrever o seu próprio trabalho usando os termos pelos quais auditores, certificadores e bancas vão cobrá-lo.

Antes de começar — pré-requisitos obrigatórios

Este sprint não funciona se faltar qualquer um dos itens abaixo. Confira antes de prosseguir:

[] Sprints 1 a 5 concluídos, com os respectivos arquivos Word de entrega salvos para consulta.

[] Pasta `internet_banking_tests` intacta, contendo no mínimo: `app.py` (refatorado com `app.config.from_object`), `config.py`, `conftest.py`, `test_ia_gerado.py`, `test_hypothesis.py`, `test_flask_client.py`, `features/transferencia.feature`, `features/steps/transferencia_steps.py`.

[] Acesso ao AVA (ambiente.virtual.ifro.edu.br) — a aula de 05-06-2026 (sexta-feira) está classificada como Dia Letivo mediado por tecnologia, e o trabalho é integralmente assíncrono no AVA.

[] Você não precisa abrir terminal, instalar nada, nem executar `pytest` neste sprint. Precisa abrir os arquivos antigos e responder por escrito. Quem quiser executar comandos de verificação opcional encontrará um bloco roxo no Passo 8 — mas é opcional.

Investigação Técnica 1 — identifique o que cada artefato faz

Antes de qualquer rotulação formal, responda em uma frase técnica para cada artefato. Não consulte o roteiro do sprint correspondente neste momento — escreva pela sua memória. O objetivo é diagnosticar quanto do seu próprio trabalho você consegue articular sem apoio externo.

- a) `test_ia_gerado.py` (Sprint 1) verifica:
- b) `conftest.py` (Sprint 2) garante:
- c) `test_hypothesis.py` (Sprint 3) verifica:
- d) A refatoração do `app.py` via `app2.py` e `config.py` (Sprint 4) viabiliza:
- e) `features/transferencia.feature` em conjunto com `transferencia_steps.py` (Sprint 5) descrevem:

Critério de avaliação: a resposta de cada item precisa ter um verbo técnico forte (verifica, garante, isola, mede, descreve, viabiliza) e mencionar pelo menos um conceito específico do sprint correspondente (caso de borda, fixture autouse, invariante universal, mutante sobrevivente, cenário Gherkin). Respostas vagas como “testa o sistema” não serão consideradas tecnicamente válidas.

Conceito: níveis de teste no BSTQB CTFL v4.0

O BSTQB CTFL v4.0 (BSTQB, 2023, Cap. 2 — *Testing Throughout the Software Development Lifecycle*) organiza os testes em quatro níveis principais. Cada nível tem um objeto de teste distinto, um conjunto típico de defeitos a detectar e uma base de teste diferente:

Teste de unidade (também chamado de teste de componente). Verifica o comportamento de uma função, método ou módulo isolado. Defeitos típicos: lógica interna, cálculos, ramificações. Base de teste: especificação do componente e código.

Teste de integração (de componentes e de sistemas). Verifica a interação entre componentes ou entre o sistema e dependências externas (banco de dados, APIs, serviços). Defeitos típicos: comunicação errada entre módulos, contratos violados, problemas de estado compartilhado. Base de teste: arquitetura e fluxos de dados.

Teste de sistema. Verifica o sistema inteiro como uma caixa fechada, contra requisitos funcionais e não funcionais. Defeitos típicos: regras de negócio violadas, fluxos end-to-end inconsistentes, comportamentos sob diferentes entradas. Base de teste: especificação de requisitos.

Teste de aceitação. Verifica se o sistema atende aos critérios definidos pelo dono do produto, pelo usuário final ou por regulações externas. Inclui quatro variantes documentadas pelo BSTQB (2023, Cap. 2): *User Acceptance Testing (UAT)*, *Operational Acceptance Testing (OAT)*, *Contractual Acceptance Testing* e *Regulatory Acceptance Testing*. Em sistemas bancários, a forma regulatória é frequentemente obrigatória por norma do Banco Central.

Bem identificadas, essas quatro categorias funcionam como uma chave de leitura para qualquer arquivo de teste do mundo real.

Investigação Técnica 2 — mapeie cada artefato a um nível

Para cada artefato, decida qual dos quatro níveis melhor o classifica. Use o procedimento de três perguntas, na ordem:

Pergunta 1: o teste invoca o sistema diretamente em código Python (importando funções e classes), ou pela camada HTTP do Flask test client? Se invoca função isolada sem subir a stack HTTP, é candidato a unidade.

Teste de Software

Pergunta 2: o teste verifica a interação entre o app.py e o banco SQLite (ou outra dependência externa)? Se sim, é candidato a integração.

Pergunta 3: o teste descreve um caminho funcional do sistema do ponto de vista de um usuário, com pré-condições, ação e resultado esperado em linguagem de negócio? Se sim, é candidato a aceitação.

Preencha a tabela abaixo no seu arquivo Word de entrega (uma linha por artefato, três colunas):

Artefato	Nível BSTQB	Justificativa em uma frase
test_ia_gerado.py (Sprint 1)		
conftest.py (Sprint 2)		
test_hypothesis.py (Sprint 3)		
app.py refatorado + test_flask_client.py (Sprint 4)		
features/transferencia.feature + steps (Sprint 5)		

Resposta técnica consolidada — leia após preencher a Investigação Técnica 2

Sua classificação do passo anterior deveria ter chegado a algo próximo do consolidado abaixo, com as devidas adaptações ao que você efetivamente produziu. Registre em seu arquivo Word qualquer ajuste que faria nas suas respostas, e justifique.

Artefato	Nível BSTQB	Justificativa
test_ia_gerado.py (Sprint 1)	Integração de componentes	Invoca os endpoints HTTP /saldo, /transferencia e /extrato por meio de requests; verifica a interação app.py ↔ SQLite. Não testa funções Python isoladas; testa a integração entre a rota Flask e a camada de persistência.
confest.py (Sprint 2)	Infraestrutura de teste (não é um nível em si)	Não é um teste — é uma fixture autouse de isolamento. Restaura o estado inicial do banco antes de cada teste, atendendo ao princípio de independência de testes (BSTQB, 2023, p. 52-53).
test_hypothesis.py (Sprint 3)	Sistema (com técnica property-based)	Verifica invariantes universais do sistema inteiro contra centenas de entradas geradas automaticamente. A invariante 3 — conservação de valor entre origem e destino — é um requisito funcional sistêmico, não local (MACIVER et al., 2019, p. 2).
app.py refatorado + test_flask_client.py (Sprint 4)	Integração refatorada para viabilizar mutation testing	Mantém o nível de integração do Sprint 1, mas substitui a interface HTTP externa pelo Flask test client por importação direta, tornando a suíte importável pelo mutmut. Não muda o que se testa; muda como se testa (JIA; HARMAN, 2011, p. 651).

Artefato	Nível BSTQB	Justificativa
test_melhorado.py (Sprint 4)	Integração com foco em capacidade detectora	Testes escritos especificamente para matar mutantes sobreviventes via cadeia Reachability–Infection–Propagation (PAPADAKIS et al., 2019, p. 285). Mantém o nível, eleva a métrica.
features/transferencia.feature + steps (Sprint 5)	Aceitação (User Acceptance Testing)	Descreve cenários em Gherkin — linguagem estruturada legível por qualquer pessoa de negócio. BDD é, por definição, uma técnica de aceitação (RAHMAN et al., 2024, p. 55-57).

Conceito: técnicas de teste no BSTQB CTFL v4.0

O Cap. 4 do BSTQB CTFL v4.0 (BSTQB, 2023) — *Test Analysis and Design* — divide as técnicas em três famílias complementares:

Caixa preta (black-box). O testador conhece apenas a especificação externa do sistema, não o código. Inclui partição de equivalência, análise de valor de fronteira (boundary value analysis), tabelas de decisão e transição de estados. A pergunta da caixa preta é: que classes de entrada geram comportamentos distintos?

Caixa branca (white-box). O testador conhece o código e seleciona casos para cobrir estruturas internas: comandos (statement coverage), ramos (branch coverage), caminhos. A pergunta da caixa branca é: que linhas e ramos do código foram efetivamente executados?

Baseada em experiência (experience-based). O testador usa intuição, conhecimento de domínio e padrões de defeitos conhecidos. Inclui error guessing e exploratory testing. A pergunta é: onde, baseado em experiência prévia, defeitos costumam se esconder?

As três famílias são complementares, não excludentes. Uma suíte robusta usa as três, e nem todas as decisões técnicas dos seus sprints cabem perfeitamente em uma única família — algumas combinam elementos de duas, como você verá a seguir.

Investigação Técnica 3 — mapeie cada decisão a uma técnica

Para cada decisão técnica abaixo, identifique a família (caixa preta, caixa branca ou baseada em experiência) e, quando aplicável, a subtécnica específica do BSTQB:

a) Caso de borda da Q8 (saldo da conta de origem exatamente igual ao valor da transferência) — Sprint 1. Que família e subtécnica essa decisão materializa?

b) As três invariantes universais formuladas no Sprint 3 e verificadas com Hypothesis. Que família caracteriza esse tipo de verificação?

c) O comando `pytest test_ia_gerado.py -v --cov=. --cov-report=term-missing` executado no Sprint 2. Que família está sendo aplicada quando se mede a coluna Missing?

d) Mutation testing com mutmut no Sprint 4 — gerar dezenas de cópias do `app.py` com pequenas alterações sintáticas e medir quantas a suíte detecta. Que família caracteriza essa técnica?

Resposta técnica consolidada — técnicas aplicadas no internet banking

Registre em seu arquivo Word qualquer divergência entre sua resposta e o consolidado abaixo, e analise honestamente onde sua articulação ficou imprecisa.

Decisão técnica	Família BSTQB	Subtécnica ou observação
Caso de borda da Q8 (saldo == valor) — Sprint 1	Caixa preta	Análise de valor de fronteira (boundary value analysis). A decisão “exatamente igual ao limite” é o exemplar canônico desta subtécnica (BSTQB, 2023, Cap. 4).
Três invariantes universais do Sprint 3 (Hypothesis)	Caixa preta — extensão property-based	Não corresponde a nenhuma das sub técnicas clássicas do BSTQB, mas é uma generalização moderna de caixa preta: em vez de selecionar um valor representativo por classe, gera-se centenas de valores automaticamente e verifica-se uma propriedade universal (MACIVER et al., 2019, p. 2).
--cov-report=term-missing (Sprint 2)	Caixa branca	Cobertura de comandos (statement coverage). A coluna Missing lista as

Teste de Software

Decisão técnica	Família BSTQB	Subtécnica ou observação
		linhas do app.py que nenhum teste executou (BSTQB, 2023, p. 52-53).
Mutmut / mutation testing (Sprint 4)	Caixa branca — verificação da qualidade da suíte	Não é uma técnica de teste no sentido estrito; é uma técnica de avaliação da suíte. Opera sobre o código (white-box) e mede se a suíte detectaria alterações sintáticas plausíveis. O Competent Programmer Hypothesis sustenta a relevância dos mutantes gerados (PAPADAKIS et al., 2019, p. 284-286).

Conceito: teste manual, teste automatizado e teste de regressão

O BSTQB CTFL v4.0 (BSTQB, 2023) distingue, ortogonal aos níveis e às técnicas, entre teste manual (executado por um humano) e teste automatizado (executado por uma ferramenta sem intervenção humana ponto a ponto). Toda a suíte que você construiu nos Sprints 1 a 5 é automatizada — você nunca abriu o navegador para testar manualmente. Isso tem consequências práticas: ganha-se velocidade, repetibilidade e capacidade de regressão, mas perde-se a sensibilidade exploratória do testador humano, que percebe inconsistências de interface que nenhum assert captura. Os autores apontam que a decisão de automatizar exige análise crítica de quando e como (COUTINHO; NASCIMENTO, 2025).

Teste de regressão é qualquer teste reexecutado para verificar se uma alteração no código não introduziu defeito em funcionalidade que antes funcionava. Quando você rodou `pytest test_ia_gerado.py -v` duas vezes seguidas no Sprint 1, a segunda execução já era, na prática, uma regressão da primeira. Quando o mutmut executou sua suíte contra dezenas de mutantes do Sprint 4, cada execução foi uma regressão sintética: “essa alteração introduzida quebraria a suíte?”. Por isso, no léxico técnico desta disciplina, o mutmut funciona como detector de regressão semântica — não regressão de um commit real, mas de mutações controladas que simulam commits plausíveis cometidos por um programador competente (JIA; HARMAN, 2011, p. 651).

Investigação Técnica 4 — formas de aceitação aplicadas ao internet banking

O BSTQB CTFL v4.0 (BSTQB, 2023, Cap. 2) cita quatro formas de teste de aceitação: User Acceptance Testing (UAT), Operational Acceptance Testing (OAT), Contractual Acceptance Testing e Regulatory Acceptance Testing. Para cada um dos três itens abaixo, responda no seu arquivo Word:

a) Que tipo de aceitação o seu features/transferencia.feature já implementa? Justifique em uma frase.

b) Se o internet banking fosse uma fintech real submetida à supervisão do Banco Central, qual das quatro formas seria obrigatória por norma, e por que seus cenários Gherkin atuais ainda não a cobrem completamente?

c) Esboce em Gherkin — sem implementar os steps — um cenário de aceitação regulatória que verifique um limite máximo diário de transferência (exemplo: R\$ 5.000,00 noturnos por padrão para PIX de pessoa física, conforme normativa do Banco Central). O esboço deve ter Funcionalidade, Cenário, Dado, Quando e Então em português, na mesma estrutura do Sprint 5.

Atividade prática opcional — comandos de verificação

Os comandos abaixo não exigem que o servidor Flask esteja rodando. Apenas inspecionam o estado atual da pasta `internet_banking_tests`. Execute somente se você tiver acesso a um terminal. Se estiver em ANP pelo celular ou tablet sem terminal, pule este bloco e siga para a Investigação Técnica 5.

Ative o ambiente virtual

Windows

```
.\venv\Scripts\activate
```

macOS / Linux

```
source venv/bin/activate
```

Liste todos os testes coletados pelo pytest, sem executar

```
pytest --collect-only
```

Liste a estrutura da pasta

Windows:

```
tree internet_banking_tests /F
```

macOS / Linux:

```
ls -R internet_banking_tests
```

Registre no seu arquivo Word: (i) quantos testes o `pytest --collect-only` enumerou no total; (ii) quantos vieram de `test_ia_gerado.py`, quantos de `test_hypothesis.py` e quantos da pasta `features/`. Esses números somam a sua suíte agregada ao final do Sprint 5.

Atenção — *troubleshooting* do bloco opcional

Se aparecer “no tests ran” ou erro de coleta, dois pontos a checar antes de tudo: (i) o terminal está dentro da pasta `internet_banking_tests`; (ii) o `venv` está ativado (você vê (`venv`) no início do prompt). Se ambos estão corretos e ainda assim falha, o problema é estrutural — provavelmente o `conftest.py` do Sprint 2 ou os arquivos do Sprint 5 estão fora do lugar. Volte ao sprint correspondente para conferir, mas isso não bloqueia o Sprint 6: a atividade prática é opcional, e a entrega central é o registro escrito das Investigações 1 a 5.

Investigação Técnica 5 — síntese epistemológica

Em três frases — não mais, não menos —, sintetize:

a) O que é uma suíte de testes para você ao final do Sprint 5? Não a definição do BSTQB; a sua definição operacional, ancorada no que você efetivamente construiu no internet banking.

b) O que mudou entre o seu olhar do Sprint 1 (“preciso que o sistema funcione”) e o seu olhar do Sprint 6 (“preciso classificar o que produzi no vocabulário do BSTQB”)?

c) Que conceito da Unidade 2 do plano de ensino (níveis, tipos, técnicas, manual vs. automatizado, regressão) você ainda não conseguiu articular com clareza, e que precisa revisitar antes da prova teórica?

Tabela epistemológica consolidada — três dimensões da sua suíte

A tabela abaixo consolida três dimensões — nível, técnica e ferramenta — em uma única visão da sua suíte ao final do Sprint 6. Cole esta tabela no seu arquivo Word de estudo. Ela é o seu mapa do território para a prova teórica e para qualquer conversa técnica futura sobre o que você produziu.

Artefato	Nível	Técnica	Ferramenta	Métrica derivada
test_ia_gerado.py	Integração	Caixa preta (boundary, equivalence)	pytest + requests / Flask test client	número de testes que passam
confest.py	infraestrutura	n/a	pytest fixture autouse	independência dos testes
test_hypothesis.py	Sistema	Caixa preta property-based	Hypothesis	número de propriedades verificadas e contraexemplos
--cov-report=term-missing	verificação da suíte	Caixa branca	pytest-cov	percentual de cobertura de linhas

Artefato	Nível	Técnica	Ferramenta	Métrica derivada
mutmut	verificação da suíte	Caixa branca	mutmut	mutation score
transferencia.feature + steps	Aceitação (UAT)	Caixa preta com linguagem natural	pytest-bdd / Gherkin	número de cenários verdes; cobertura semântica

Reflexão do Sprint 6 — registre em seu arquivo Word antes de avançar

Responda com suas próprias palavras. Não existe resposta certa. O que importa é que o texto revele o seu raciocínio real durante a execução.

1. Antes deste sprint, se um auditor chegasse à sua mesa e perguntasse “que tipo de teste está escrito neste arquivo test_ia_gerado.py?”, você teria conseguido responder em vocabulário formal? E agora, ao final do Sprint 6?

2. Olhe a tabela epistemológica consolidada. Qual linha você teve mais dificuldade para preencher, e por quê?

3. O BSTQB CTFL v4.0 organiza os testes em níveis (Cap. 2) e técnicas (Cap. 4). Em uma frase — usando exemplos da sua suíte do internet banking, não definições genéricas —, qual é a diferença entre nível e técnica?

4. Se você fosse contratado amanhã como testador júnior em uma fintech e seu gerente pedisse para descrever a estratégia de teste do produto em uma frase usando o vocabulário do BSTQB, o que você diria? Use a sua suíte do internet banking como referência.

5. Da Unidade 2 do plano de ensino (níveis: unidade, integração, sistema, aceitação; técnicas: caixa preta, caixa branca, baseada em experiência; teste manual vs. automatizado; teste de regressão), há algum item que sua suíte do internet banking não exemplifica empiricamente? Qual, e como você o exemplificaria com um novo arquivo se tivesse uma semana extra?

6. Em uma linha: o que você ganha em poder descritivo agora que sabe o nome formal do que produziu?

Entrega no AVA

Arquivo Word ou PDF único contendo, na ordem:

- 1) Suas respostas completas à Investigação Técnica 1 (cinco itens, uma frase técnica cada).
- 2) Sua tabela preenchida da Investigação Técnica 2 (mapeamento artefato → nível BSTQB).
- 3) Quaisquer ajustes que você faria às suas respostas da Investigação 2 após ler a resposta consolidada, com justificativa em uma frase.
- 4) Suas respostas à Investigação Técnica 3 (decisão técnica → família BSTQB → subtécnica).
- 5) Suas respostas completas à Investigação Técnica 4 (formas de aceitação), incluindo o esboço Gherkin do item c.
- 6) Registro dos números do pytest --collect-only — apenas se executou o bloco opcional.
- 7) Suas três frases da Investigação Técnica 5 (síntese epistemológica).
- 8) Cópia da Tabela epistemológica consolidada, com qualquer ajuste que sua execução real precisar.
- 9) Reflexão final do Sprint 6 (seis perguntas).

Pontuação: até 5 pontos. A nota considera: (i) ancoragem nos autores indicados nas referências; (ii) coerência interna entre suas respostas; (iii) ancoragem específica no cenário do internet banking e nos arquivos efetivamente produzidos nos Sprints 1 a 5 — não respostas genéricas; (iv) honestidade no item de revisão da Investigação 2 — reconhecer ajustes vale mais que esconder divergências; (v) profundidade da síntese epistemológica do item 5 e da reflexão final.

Referências Bibliográficas

ANICHE, **Maurício**. *Testes automatizados de software: um guia prático*. 1. ed. São Paulo: Casa do Código, 2015.

BSTQB — BRAZILIAN SOFTWARE TESTING QUALIFICATIONS BOARD. *Certified Tester Foundation Level Syllabus*: versão 4.0. [S. l.]: ISTQB; BSTQB, 2023. Disponível em: https://www.bstqb.online/files/syllabus_ctfl_4.0br.pdf. Acesso em: maio 2026.

COUTINHO, N. de M.; NASCIMENTO, E. L. B. do. Desafios e benefícios da implementação de testes automatizados em empresas de software. *Cuadernos de Educación y Desarrollo*, v. 17, n. 4, e8221, p. 1-19, 2025. DOI: <https://doi.org/10.55905/cuadv17n4-176>. Disponível em: <https://ojs.cuadernoseducacion.com/ojs/index.php/ced/article/view/8221>. Acesso em: maio 2026.

DELAMARO, Márcio; MALDONADO, José Carlos; JINO, Mário. *Introdução ao teste de software*. Rio de Janeiro: Campus, 2007.

JIA, Y.; HARMAN, M. An analysis and survey of the development of mutation testing. *IEEE Transactions on Software Engineering*, v. 37, n. 5, p. 649-678, set./out. 2011. DOI: 10.1109/TSE.2010.62. Disponível em: <https://ieeexplore.ieee.org/document/5487526>. Acesso em: maio 2026.

MACIVER, D. R.; HATFIELD-DODDS, Z. et al. Hypothesis: a new approach to property-based testing. *Journal of Open Source Software*, v. 4, n. 43, p. 1891, 2019. DOI: 10.21105/joss.01891. Disponível em: <https://joss.theoj.org/papers/10.21105/joss.01891>. Acesso em: maio 2026.

PAPADAKIS, M.; KINTIS, M.; ZHANG, J.; JIA, Y.; LE TRAON, Y.; HARMAN, M. Mutation testing advances: an analysis and survey. *Advances in Computers*, v. 112, p. 275-378, 2019. DOI: 10.1016/bs.adcom.2018.03.015. Disponível em: <https://www.sciencedirect.com/science/article/abs/pii/S0065245818300305>. Acesso em: maio 2026.

RAHMAN, Md. S. et al. Evaluating the impact of test-driven development on software quality enhancement. *International Journal of Mathematical Sciences and Computing*, v. 10, n. 3, p. 51-76, 2024. DOI: 10.5815/ijmsc.2024.03.05. Disponível em: <https://www.mecs-press.org/ijmsc/ijmsc-v10-n3/IJMSC-V10-N3-5.pdf>. Acesso em: maio 2026.